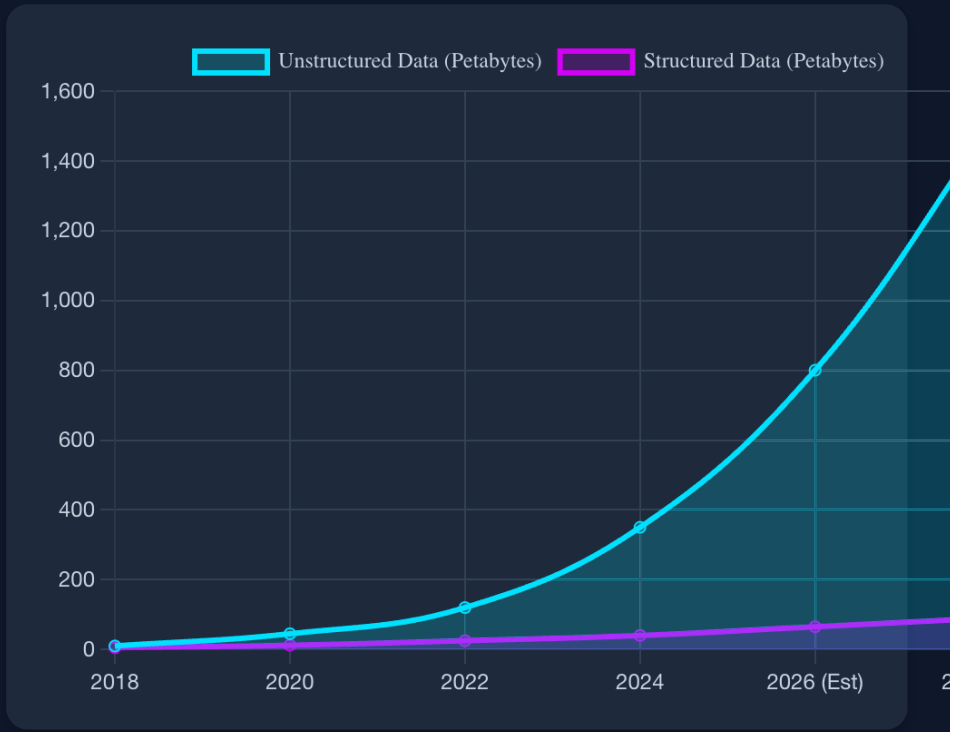
Unstructured Data in Snowflake

An In-Depth Engineering Report & Implementation Demos

The Data Tsunami

Historically, data warehouses were rigid, designed solely for tabular, structured data. However, over 80% of enterprise data today is unstructured—PDFs, images, audio, video, and raw text. Ignoring this wealth of information leaves massive analytical potential on the table.

Snowflake has fundamentally shifted this paradigm. By extending its core architecture to natively support, govern, and process unstructured files, organizations can now bring their code to the data, eliminating brittle ETL pipelines and external processing clusters.



Core Architecture: Storage & Access

Processing unstructured data begins with secure storage and access. Snowflake utilizes **Internal and External Stages** to house files. To make these files queryable and actionable, Snowflake introduces **Directory Tables** and various secure URL types.

Directory Tables

Implicit, built-in tables layered on top of stages. They provide a catalog of staged files, capturing file URIs, sizes, and MD5 hashes, enabling standard SQL queries over file metadata.

Scoped URLs

Encoded URLs that grant temporary access to a specific file without requiring privileges on the underlying stage. Perfect for secure sharing within Snowflake UI or custom apps.

Pre-signed URLs

URLs generated for downloading or accessing files over HTTPS via third-party tools or external applications, authenticated via the cloud provider (AWS/Azure/GCP) seamlessly.

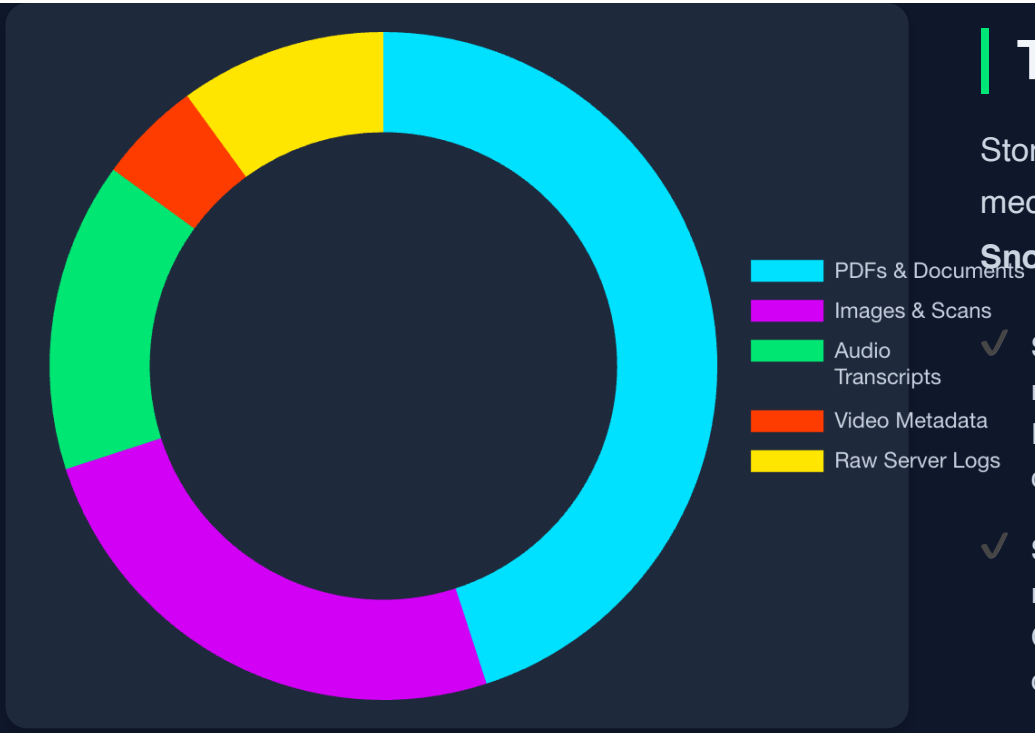
Demo 1: Ingestion & Governance Setup

Creating an internal stage with directory table enabled, uploading a file, and querying its metadata.

```sql
-- 1. Create a stage configured for unstructured data
CREATE STAGE pdf_contracts_stage
  DIRECTORY = (ENABLE = TRUE)
  ENCRYPTION = (TYPE = 'SNOWFLAKE_SSE');

-- 2. Query the directory table to see available files
SELECT RELATIVE_PATH, SIZE, FILE_URL
FROM DIRECTORY(@pdf_contracts_stage);

-- 3. Generate a secure scoped URL for a specific user role
SELECT BUILD_SCOPED_FILE_URL(@pdf_contracts_stage, 'vendor_contract_2026.pdf');
```



The Processing Engines

Storing files is only step one. Snowflake provides two primary mechanisms for natively processing this data where it lives: **Snowpark** and **Snowpark Container Services (SPCS)**.

- **Snowpark (Python/Java/Scala UDFs):** Ideal for lightweight to medium processing. Read a file stream, extract text using PyPDF2, or run standard NLP libraries directly within Snowflake's compute warehouse.
- **Snowpark Container Services:** For heavy-duty processing requiring custom runtimes, GPUs, or complex dependencies (e.g., OpenCV for video processing, massive LLMs). Deploy Docker containers directly inside Snowflake's security perimeter.

Demo 2: Snowpark Python UDF Pipeline

In this demo workflow, we deploy a Python User-Defined Function (UDF) to extract text from PDF invoices stored in our internal stage. The UDF leverages the `PyPDF2` library provided via Snowflake's Anaconda integration.

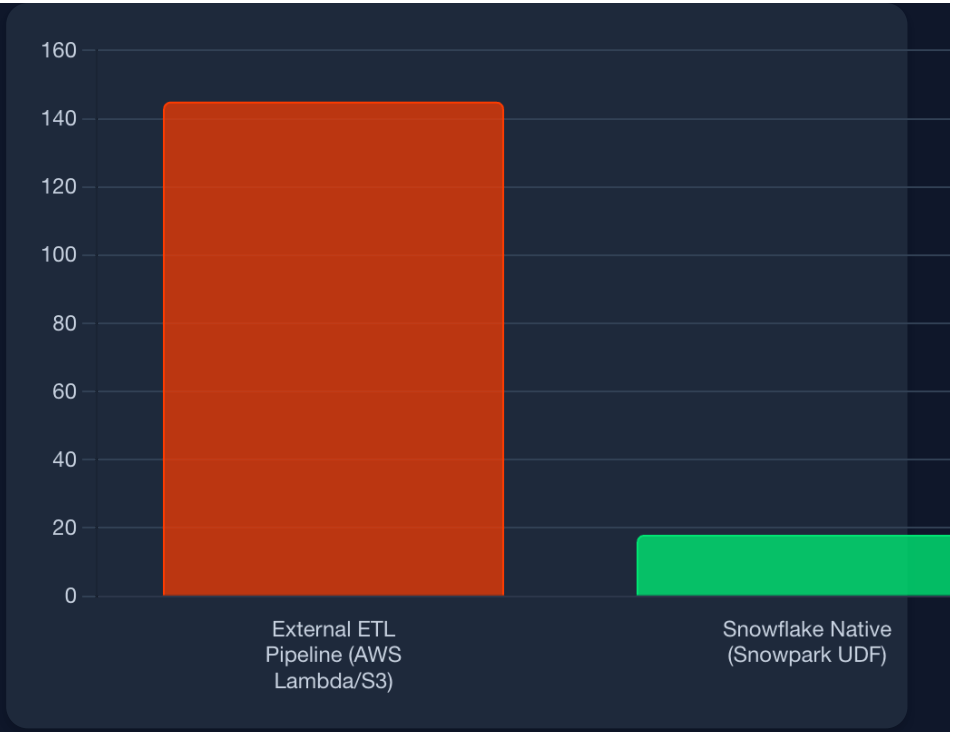
```
-- Create a Python UDF to parse PDFs dynamically
CREATE OR REPLACE FUNCTION extract_pdf_text(file_path STRING)
RETURNS STRING
LANGUAGE PYTHON
RUNTIME_VERSION = '3.10'
PACKAGES = ('pypdf2', 'snowflake-snowpark-python')
HANDLER = 'process_file'
AS
$$ import PyPDF2 from snowflake.snowpark.files import SnowflakeFile def process_file(file_path): #
SnowflakeFile securely streams the file from the stage with SnowflakeFile.open(file_path, 'rb') as f: reader =
PyPDF2.PdfReader(f) text = "" for page in reader.pages: text += page.extract_text() + "\n" return text $$;
```

```
-- Execute the UDF against all files in the directory table
CREATE TABLE parsed_contracts AS
SELECT
  RELATIVE_PATH AS filename,
  extract_pdf_text(BUILD_SCOPED_FILE_URL('@pdf_contracts_stage', RELATIVE_PATH)) AS extracted_content
FROM DIRECTORY(@pdf_contracts_stage);
```

Performance Impact

Bringing processing to the data eliminates massive network I/O bottlenecks and structural complexities associated with traditional external ETL pipelines (like AWS Lambda or external Spark clusters).

By leveraging Snowflake's highly parallel virtual warehouses and native Snowpark environments, organizations see dramatic reductions in end-to-end processing time, specifically when parsing hundreds of thousands of small-to-medium unstructured documents.



Document AI & Cortex: The Final Frontier

While extracting raw text is powerful, extracting *meaning* is the ultimate goal. Snowflake **Document AI** and **Cortex AI** natively embed Large Language Models (LLMs) directly into the SQL layer, allowing you to ask questions of your unstructured data instantly.

Demo 3: Cortex AI Extraction

Using native SQL LLM functions to extract specific data points from the raw text we generated in Demo 2.

Cortex Capabilities

```sql
-- Query the parsed text using Cortex LLM to find the
total contract value
SELECT
  filename,
  SNOWFLAKE.CORTEX.EXTRACT_ANSWER(
    extracted_content,
    'What is the total monetary value of this
contract?'
  ) AS contract_value
FROM parsed_contracts;
```

- **EXTRACT_ANSWER:**QA against documents.
- **SUMMARIZE:**Condense 100-page PDFs instantly.
- **SENTIMENT:**Analyze audio transcripts.
- **TRANSLATE:**Multi-lingual document normalization.